Suggested Teaching Guidelines for

Object Oriented Programming with Java

PGCP-AC, August 2026

Duration: 150 Hours: 60 Theory (T) + 60 Lab (L) + 30 Self Learning (SL)

Objective: To reinforce knowledge of Object-Oriented Programming concepts using Core Java.

Prerequisites: Basic programming knowledge, OOP concepts, and SQL

Evaluation method: Theory exam – 40%, Lab exam – 40%, Internal Assessment – 20%

List of Course Material

Text book:

1. Object-Oriented Programming and Data Structures using Java, 2026 edition, Cay S. Horstmann / Wiley

Reference book:

1. Java The Complete Reference, 9th edition, Herbert Schildt / McGraw Hill
2. Murach's Java Programming, 4th edition, Joel Murach
3. Head First Java, 3rd edition, Kathy Sierra, Bert Bates, Trisha Gee / SPD

Note: *This module adopts Java 17 (LTS) as the programming standard for all sessions, lab work and evaluations.*

Session 1: Introduction to Java (2T + 2SL)

- Introduction to java
- Features of java
- JVM Architecture
- JDK and its usage
- Structure of java class
- Working with data types: Primitive data types
- Java Tokens: Keywords, Identifiers, Literals

Sessions 2 & 3: Basic programming concepts (4T + 4L + 2SL)

- Declaring variables and methods
- Data type compatibility
- Operators
- Conditional Statements (switch expression: Java 14+)
- Control Statements
- Arrays 1-D and multidimensional array

Lab:

- Get yourself acquainted with java environment and Setup your IDE

Suggested Teaching Guidelines for

Object Oriented Programming with Java

PGCP-AC, August 2026

- Write a program to calculate the total bill amount after applying a discount based on the purchase amount
- Write a menu-driven calculator using the Java 14+ switch expression to perform: Addition, Subtraction, Multiplication & Division
- Write a program to find the factorial of a number using a loop.
- Write a program to check whether a number is an Armstrong number.
- Write a program to generate the Fibonacci series up to N terms.
- Print different patterns of asterisk (*) using loops (e.g. triangle of *)
- Write a program to find principal diagonal sum of a matrix

Session 4: Object Oriented Programming Concepts (2T + 2L+ 2SL)

- Introduction to OOP
- Classes and Objects
- OOP principles
- Encapsulation, Abstraction, Inheritance and Polymorphism

Lab:

- Create a Student class with attributes such as roll number, name, and marks.
- Create objects of the class and display student details.

Session 5: Static Members and Object References (2T + 2L)

- Static variables and methods
- Accessing static variables and methods of different class
- Introduction to reference data types
- Reference variables and methods
- Difference between reference data types and primitive data types
- Difference between reference variable and static variable

Lab:

- Create a class Student with:
  - Instance variables: rollNo, name
  - Static variable: collegeName
- Demonstrate that all objects share the same static variable.
- Create a static method to display the college name.
- Create a class BankAccount with:
  - Account number and balance as instance variables.
  - Bank name as a static variable.
- Demonstrate that changing the bank name affects all account objects.

Suggested Teaching Guidelines for

Object Oriented Programming with Java

PGCP-AC, August 2026

Session 6: Object Creation, Memory Management and Records (2T + 2L + 2SL)

- Heap memory and Stack memory
- Constructors, initializing reference variables using constructors.
- Pass by value v/s pass by reference.
- Re-assigning a reference variable.
- Passing reference variable to method
- Initializing reference variable of different class
- Records

Lab:

- Design a simple application (ex. Student Management System) using:
  - Classes and constructors
  - Object references
  - Association between classes
  - Records for storing immutable student details

Session 7: Inheritance (2T + 2L + 2SL)

- Inheritance: single & multilevel
- Inheritance: Hierarchical
- Association, Aggregation and Composition
- Polymorphism: Compile time and runtime polymorphism
- Rules of overriding and overloading of methods
- super and this keyword

Lab:

- Design a Employee Management System with:
  - Person → Employee → Manager hierarchy.
  - Department–Employee aggregation.
  - Address–Employee composition.
  - Overloaded methods for salary calculation.
  - Overridden methods for displaying employee details.

Session 8: Packages, Access Control and Type Casting (2T + 2L)

- Access modifiers (public, private, protected and default)
- Packages and import statements
- Static imports
- Constructor chaining (with and without packages)
- Accessing protected variables and methods outside the package
- Upcasting and downcasting of a reference variable
- Instance of operator

Suggested Teaching Guidelines for

Object Oriented Programming with Java

PGCP-AC, August 2026

Lab:

- Design a University Management System using multiple packages:
  - Package for Person, Student, and Faculty
  - Package for utility classes
- Demonstrate:
  - Access modifiers
  - Constructor chaining
  - Upcasting and downcasting
  - instanceof checks before casting

Session 9: Wrapper Classes and String Class (2T + 4L+ 4SL)

- Wrapper classes and constant pools
- String class, StringBuffer & StringBuilder class
- String pool

Lab:

- Demonstrate boxing and unboxing
- Demonstrate autoboxing and auto-unboxing
- Use wrapper utility methods such as:
  - parseInt()
  - parseDouble()
  - valueOf()
  - toString()
  - compareTo()
- Create String objects using:
  - String literals
  - new String()
- Compare strings using:
  - ==
  - equals()
- Observe the behavior of String Constant Pool.
- Perform operations using:
  - length()
  - charAt()
  - substring()
  - indexOf()
  - contains()
  - replace()
  - split()
  - toUpperCase()
  - toLowerCase()
  - trim()

- Create programs demonstrating:
  - append()
  - insert()
  - delete()
  - replace()
  - reverse()
- Compare the behavior of String, StringBuffer, and StringBuilder.

Session 10: Enumeration and Java Class Features (2T + 2L + 2SL)

- Enumeration
- Sealed class
- Final variables and final methods
- Final class
- Pattern matching
- Inner Class (Regular, Method local, Anonymous & static inner class)

Lab:

- Develop a Employee Management System that:
  - Uses an enum for employee departments.
  - Uses a sealed class hierarchy for employee types.
  - Uses pattern matching to process different employee objects.
  - Uses inner classes to manage employee details and reports.

Sessions 11 & 12: Abstract Class and Interfaces (4T + 2L + 2SL)

- Abstract class and abstract methods
- Interface (implementing multiple interfaces)
- New interface features (Java 8 &11)

Lab:

- Design a Smart Device Management System where:
  - Device is an abstract class
  - Interfaces such as Connectable, Rechargeable, and Upgradeable define device capabilities
  - Different devices implement one or more interfaces
  - Use default methods for common functionality

Session 13: Garbage Collection (2T + 2L)

- Garbage collection in java
- Requesting JVM to run garbage collection

- Different ways to make object eligible for garbage collection: (Nulling a reference variable, Re-assigning a reference variable & island of isolation)
- Finalize method

Lab:

- Override finalize method to understand behavior of JVM garbage collector
- Study when to prefer an abstract class over an interface

Sessions 14 & 15: Exception Handling (4T + 4L + 2SL)

- Exception hierarchy, Errors, Checked and un-checked exceptions.
- Exception propagation
- try-catch-finally block, throws clause and throw keyword.
- Multi catch block.
- Creating user defined checked and unchecked exceptions.

Lab:

- Develop a simple banking application that:
  - Validates account operations
  - Uses custom checked exceptions for insufficient balance
  - Uses custom unchecked exceptions for invalid input
  - Demonstrates exception propagation
  - Uses throw, throws, try-catch-finally, and multi-catch blocks

Sessions 16: File IO (2T + 2L)

- Brief introduction to InputStream, OutputStream
- Reader and Writer interfaces
- NIO package Overview
- Serialization and de-serialization

Lab:

- Create a program to:
  - Write data to a file using FileOutputStream, FileWriter and FileWriter
  - Read data from a file using FileInputStream, FileReader and BufferedReader
- Serialize an object and store it in a file.
- Read the serialized object from the file.
- Display the restored object's details.
- Create a serializable class containing transient fields.
- Observe which fields are serialized and which are skipped.

Suggested Teaching Guidelines for

Object Oriented Programming with Java

PGCP-AC, August 2026

Session 17: Object Class & Date Time API (2T + 2L)

- LocalDate and LocalDateTime APi
- Converting LocalDate to String and String to LocalDate
- Object Class: Overriding to String, equals & hashcode method

Lab:

- Develop a Employee Attendance System that:
  - Stores employee details and joining dates.
  - Calculates employee experience using LocalDate.
  - Uses overridden toString(), equals(), and hashCode() methods.

Session 18: Generics (2T + 2SL)

- Introduction to generics
- Generic classes
- Generic methods
- Wild cards (upper and lower)

Sessions 19: Collection Framework (2T)

- Introduction to collections
- Collection hierarchy
- List, Queue, Set and Map Collections

Sessions 20 & 21: List (4T + 4L)

- List Collection:
  - ArrayList,
  - LinkedList
  - Vector
- Collections class

Lab:

- Store student details in an ArrayList.
- Implement Student Management System:
  - Add Student
  - Search Student by Roll Number
  - Update Student Details
  - Delete Student
  - Display All Students
- Implement a Task Management System
- Use a LinkedList to maintain tasks.
- Support:
  - Add task

- Remove task
- Mark task as completed
- Display pending tasks

Sessions 22 & 23: Set and Map (4T + 4L)

- Hashing Internals
  - Introduction to hashing and hash functions
  - hashCode() contract in Java
  - The equals() and hashCode() relationship
  - Internal bucket structure (Java 8+)
  - Load factor and initial capacity
  - Rehashing and resizing
- Set Collection:
  - HashSet, LinkedHashSet & TreeSet collection
- Map Collection:
  - HashTable, HashMap, LinkedHashMap & TreeMap classes

Lab:

- Demonstrate collision: create two objects with the same hashCode() but different equals() and verify both are stored
- Create an Employee HashSet collection and override equals & hashCode methods to understand how the set maintains uniqueness using these methods.
- Create a Sample class to understand generic assignments using “? Extends Some Class” , “? Super some class ” and “?”
- Implememt Library Management System using HashMap to maintain: Book ID → Book Details and Implement:
  - Add Book
  - Search Book
  - Update Book Details
  - Delete Book
- Implement Student Ranking System:
  - Store student marks using TreeMap.
  - Display records sorted by roll number.
  - Generate rank list based on marks.
- Implement Unique Visitor Tracker:
  - Use HashSet to maintain unique visitor IDs.
  - Display total unique visitors and prevent duplicate entries.

Session 24: Sorting Collections (2T + 2L + 2SL)

- Introduction to Sorting in Collections Framework
- Natural Sorting using the Comparable Interface

- Custom Sorting using the Comparator Interface
  - Sorting Objects Based on Multiple Attributes
  - Anonymous Class Implementation of Comparator
  - Comparator using Lambda Expressions
- Comparator Helper Methods (Java 8+)

Lab:

- In Student Merit List System, Generate the following reports:
  - Sort by Roll Number (Ascending)
  - Sort by Name (Ascending)
  - Generate a Merit List by Marks (Descending).
  - Sort by Department (Ascending) and then Name (Ascending)
  - Sort by Department (Ascending) and then Marks (Descending)
  - Sort by Marks (Descending) and then Roll Number (Ascending)
  - Sort by Department (Ascending), Marks (Descending), and then Name (Ascending)

Sessions 25, 26 & 27: Functional Programming and Stream API (6T + 8L+ 2SL)

- Introduction to Functional Programming
- Functional Interfaces
- Built-in Functional Interfaces
  - Predicate
  - Function
  - Consumer
  - Supplier
- Lambda Expression Syntax
- Method References
  - Static Method Reference
  - Instance Method Reference
  - Constructor Reference
- Benefits and Applications of Lambda Expressions
- Lambda Expressions
- Implementation of Java 8 stream API
- Intermediate Operations
  - filter(), map(), flatMap()
  - sorted()
  - distinct(), , limit(), skip()
- Terminal Operations
  - forEach()
  - collect(), count(), reduce()
  - min(), max()
  - findFirst()
  - anyMatch(), allMatch(), noneMatch()

- Collectors Utility Class
  - toList(), toSet(), groupingBy(), partitioningBy(), joining()
- Parallel Streams
- Advantages of Stream API over Traditional Iteration

Lab:

- Create custom Functional Interfaces and implement them using Lambda Expressions.
- Use Predicate, Function, Consumer, and Supplier interfaces in different programs.
- Implement sorting and searching using Lambda Expressions.
- Demonstrate Method References and Constructor References.
- Create Streams from Collections and Arrays.
- Perform filtering and transformation operations using Stream API.
- Implement aggregate operations using reduce(), count(), min(), and max().
- Group and partition data using Collectors.
- Develop a mini application to process employee/student records using Stream API.

Session 28: Multithreading and Synchronization (2T+ 4L+ 2SL)

- MultiThreading : Thread class and Runnable Interface
- sleep, join, yield, setPriority, getPrioritymethods.
- ThreadGroup class
- Synchronization
- Introduction to Wait, notify and notifyAllmethods.
- Overview of Concurrent collections: Concurrent HashMap
- Metadata & Reflection

Lab:

- Create multiple threads using Thread class and Runnable interfaces.
- Assign same task and different task to multiple threads.
- Understand sleep, join, yield methods.
- Implement wait, notify and notifyAll methods.
- Demonstrate how to share thread local data between multiple threads.
- Invoke private methods of some other class using reflection.
- Create multiple threads using anonymous inner classes.
- Create multiple threads using lambda expressions.

Sessions 29 & 30: JDBC & Transaction Management (4T+ 6L+ 2SL)

- Introduction to JDBC API
- JDBC Architecture
- JDBC Drivers
- JDBC Classes & Interfaces
- Driver, Connection, Statement, PreparedStatement, ResultSet

Suggested Teaching Guidelines for

Object Oriented Programming with Java

PGCP-AC, August 2026

- Stored procedures and functions Invocation
- Introduction to Database Metadata
- SQL Injection overview and prevention
- Design Pattern: Data Access Object Pattern

Lab:

- Perform Database CRUD operations using JDBC Classes